Hello NSTUian

Master Project Documentation

Document Type: Software Requirements Specification (SRS), System Architecture, Database Design, UI/UX Specification, Security Rules, API Plan, and AI Development Guide
Project Version: 1.0
Project Status: Planning and specification
Primary Audience: Non-technical stakeholders, students, developers, testers, and AI coding agents
Primary Language: English for the website interface and technical documentation
Project Name: Hello NSTUian
Project Purpose: A public, mobile-first academic question bank for Noakhali Science and Technology University (NSTU).

---

1. Project Overview

1.1 What is Hello NSTUian?

Hello NSTUian is a web-based academic question bank that helps NSTU students find previous examination questions easily.

Students will be able to browse departments, university admission sessions, semesters, courses, and examination questions. They will not need to create an account to view or download approved question papers.

Students with verified NSTU EduMail accounts will be able to contribute new question papers and request corrections to existing papers.

The system will also have an administrative dashboard for managing academic information, reviewing changes, and controlling department-specific permissions.

1.2 The problem this project solves

Academic questions are often scattered across messaging groups, social media posts, personal devices, and different student communities. Finding a question from a particular course, semester, or examination year can therefore be difficult.

Hello NSTUian will organize these resources in a single searchable platform.

The project aims to:

- Make academic question papers easy to find.

- Provide free public access to approved questions.
- Organize papers by department, admission session, semester, course, and examination.
- Allow verified NSTU students to contribute questions.
- Prevent unauthorized changes to existing questions.
- Distribute administrative responsibilities among department-specific admins.
- Provide a responsive experience on mobile phones, tablets, and computers.

1.3 Project goals

1. Create a user-friendly academic resource platform.
2. Make question discovery fast and straightforward.
3. Support different question-paper formats.
4. Minimize unnecessary administrative work through automatic publication of valid new submissions.
5. Protect existing published papers through an approval workflow for modifications.
6. Enforce secure, department-based administrative access.
7. Build a maintainable system that can support future expansion.

1.4 Project scope

Included in the initial version

- Public landing page.
- Alphabetically sorted department directory.
- Search functionality.
- University session and academic navigation.
- Eight-semester structure for each department's supported program.
- Course and examination question management.
- CT-1, CT-2, CT-3, and multiple CT sets.
- Semester/final examination questions.
- Question-paper upload, preview, and download.

- EduMail verification for contributors.
- Automatic publication of eligible new questions.
- Duplicate detection.
- Question correction and replacement requests.
- Super Admin and Department Admin roles.
- Administrative dashboard.
- Audit logs and change history.
- Responsive glassmorphism interface.

Not required for the initial version

The following features are outside the mandatory initial scope:

- Online examination or quiz-taking system.
- Automated grading.
- Student attendance management.
- Payment gateway.
- Mandatory login for public question access.
- AI-generated answers to examination questions.
- Mobile applications for Android or iOS.

These may be considered separately in future versions.

---

2. People Who Will Use the System

2.1 Public visitor

A public visitor is anyone who opens Hello NSTUian.

They can:

- Browse all published departments.
- Search for departments and courses.
- Browse academic sessions and semesters.

- View published question papers.
- Download available question files.

They cannot upload questions, submit correction requests, or access administrative features without signing in and meeting the required verification rules.

2.2 Verified NSTU contributor

A contributor is an authenticated user whose NSTU EduMail ownership has been verified.

They can:

- Use all public features.
- Upload new question papers for any NSTU department.
- Submit a correction request for an existing paper.
- Propose a replacement file for an existing paper.
- View the status of their submissions and requests.

A contributor does not automatically receive administrative permissions just because their email is verified.

Contributors may submit questions for departments other than their own. Their department membership does not restrict which department's questions they can contribute.

2.3 Department Admin

A Department Admin is responsible for managing a specific department.

They can:

- Manage questions within their assigned department.
- Review correction and replacement requests for that department.
- Edit, replace, approve, reject, or archive question papers within their permissions.
- Manage permitted academic records within their department.
- View relevant audit history.

They cannot modify another department's records or change global system settings.

2.4 Super Admin

The Super Admin has system-wide administrative authority.

They can:

- Manage all departments and academic records.
- Manage question papers across all departments.
- Review and resolve submissions and correction requests.
- Create and manage Department Admin accounts and assignments.
- Configure system-wide settings.
- Access global audit logs.
- Suspend accounts and investigate suspicious activity.

A Super Admin should use a separate administrative account with strong authentication and appropriate session security.

---

3. Core Business Rules

These rules are mandatory and take precedence over implementation preferences.

BR-01: Public access

Viewing and downloading approved, published question papers must not require login.

BR-02: Contributor verification

Uploading questions or requesting changes requires an authenticated account with a verified, eligible NSTU EduMail address.

BR-03: First-time question upload

A genuinely new question paper submitted by a verified user should be published automatically if it passes all required validation and duplicate checks.

Manual admin approval is not required for a valid new paper.

BR-04: Existing question modification

Any user-requested edit, correction, or replacement of an existing published question paper requires admin approval.

The existing published version must remain available while the proposed change is under review.

BR-05: Duplicate protection

A submission that appears to duplicate an existing question must not be automatically published as a new record. It should be flagged for review or handled through the correction workflow.

BR-06: Admin-only editing

Only authorized admins can directly modify an official published question paper.

A regular contributor can propose changes but cannot overwrite official content.

BR-07: Department-level authorization

A Department Admin may modify only records belonging to their assigned department.

The backend must enforce this restriction on every relevant API request.

BR-08: Global authorization

A Super Admin may manage records belonging to any department.

BR-09: Department-neutral contributions

A verified contributor may submit a new question or correction for any department, regardless of their own department.

BR-10: File safety

Every uploaded file must pass security and file-type validation before publication or delivery.

BR-11: Change history

Administrative edits, approvals, rejections, replacements, and archive actions must be recorded in an audit log.

BR-12: No automatic correction

A correction request must never change an existing official question until an authorized admin approves the change.

---

4. Website Navigation and Academic Structure

4.1 Public website navigation

The main navigation should provide access to:

- Home
- Departments
- Search
- About
- Contribute Questions
- Login or Account
- Admin Dashboard, visible only to authorized administrators

The landing page must prioritize department discovery and question search.

4.2 Department directory

Departments must be displayed in alphabetical A–Z order based on their official display names.

Each department should have:

- Official name.
- Unique department ID.
- Optional abbreviation.
- Optional department image or icon.
- Available academic programs or curricula.
- Number of available courses or question papers, where applicable.

Department names and IDs must be managed through the database rather than hardcoded throughout the application.

4.3 Academic hierarchy

The default browsing hierarchy is:

Department → University Session → Semester → Course → Examination → Question Paper

Each department/program structure must support eight semesters.

A semester contains multiple courses, and each course can have question papers from multiple examination years or sessions.

Important distinction

A university admission session and an examination session are different concepts.

For example:

- University session: 2022–2023.
- Semester: Semester 3.
- Course: Database Management Systems.
- Examination session: 2025.
- Examination type: CT-2, Set B.

The system must store these values separately.

A student admitted in a particular session may have courses and examination papers from several different years.

4.4 Course information

Each course should support:

- Course code.
- Course title.
- Department or program.
- Semester.
- Optional credit value.
- Optional course description.
- Active/inactive status.

Course codes may not be globally unique across all departments. The database should use a suitable composite uniqueness rule or a separate internal course ID.

4.5 Examination categories

The system must support at least these categories:

Continuous Test (CT)

- CT-1
- CT-2
- CT-3

Each CT can contain multiple sets, including Set A, Set B, Set C, and additional sets.

Semester Examination

- Midterm, if used by a department.
- Final or semester examination.
- Other configured semester examination types, if needed.

The data model should allow configurable examination categories without requiring a database redesign whenever a new category is introduced.

---

5. Question Paper Management

5.1 Question paper information

Every official question paper should have the following information where applicable:

- Unique question-paper ID.
- Department.
- University session or academic batch, if applicable.
- Semester.
- Course.
- Examination type.
- CT number, when the examination type is CT.
- Set name, when applicable.
- Examination year/session.
- Optional examination date.
- Optional title or description.
- Original filename.
- Stored file location.
- File MIME type.
- File size.
- Uploader.

- Publication status.
- Creation timestamp.
- Last modification timestamp.
- Approval or review history, when applicable.

Some metadata may be optional if it is not known, but the system should require enough information to place a paper in the correct academic location.

5.2 Supported file formats

The system should support:

- PNG
- JPG and JPEG
- PDF
- DOC and DOCX
- HEIC and HEIF
- WebP
- Other explicitly supported file formats that can be safely stored and delivered.

The term "any format" must not mean accepting arbitrary executable or dangerous files.

File handling rules

1. Preserve the original file when it is safe to store.
2. Validate the extension and inspect the actual file content.
3. Set configurable upload-size limits.
4. Generate browser-compatible previews where possible.
5. Convert HEIC/HEIF to a supported preview format when necessary.
6. Use PDF preview for PDF files where supported.
7. Provide downloads for formats that cannot be previewed.
8. Never execute uploaded content.
9. Do not allow uploaded HTML or JavaScript to run on the application's origin.

10. Reject or quarantine files that fail validation.

File conversion must not be required for every supported format. Unsupported previews can fall back to a safe download option.

5.3 Public question display

A question detail page should show:

- Department and course information.
- Examination type and set.
- Examination year/session.
- File preview, when supported.
- Download action.
- Relevant metadata.
- A Request Correction action for authenticated, verified contributors.

Only published question papers should appear in public search results.

---

6. Upload and Review Workflows

6.1 First-time upload workflow

1. A user opens the contribution feature.
2. The user signs in or creates an account.
3. The user verifies ownership of an eligible NSTU EduMail address.
4. The user selects the department, session/program context, semester, course, examination type, and relevant metadata.
5. The user uploads the question paper.
6. The backend validates the user's account, metadata, file, and file safety.
7. The backend checks whether the submission may duplicate an existing paper.
8. If the submission is valid and genuinely new, it is automatically published.
9. The system records the uploader, timestamps, file metadata, and publication event.
10. The contributor receives a success message.

Automatic publication requirements

Automatic publication is permitted only if:

- The user is authenticated.
- EduMail verification is valid.
- The account is not suspended or blocked from contributing.
- The selected department, course, and academic references are valid.
- The file passes security validation.
- Required metadata is present.
- No unresolved duplicate warning exists.

A new submission that fails these checks must not become publicly available.

A possible duplicate should enter a review workflow rather than create an unverified duplicate record.

6.2 Duplicate detection

Duplicate detection should use several signals:

1. Same department.
2. Same course.
3. Same examination type.
4. Same CT number and set, when applicable.
5. Same examination year/session.
6. Identical file hash.
7. Similar metadata or file content, where practical.

An identical file hash is strong evidence that two uploads contain the same file. Different hashes do not prove that two papers are different because the same paper may be scanned, compressed, or photographed differently.

For the first version, use deterministic metadata matching and file hashes. More advanced image similarity or OCR-based matching may be added later.

If the system cannot confidently determine whether a paper is new, it should flag the submission for review instead of silently overwriting or duplicating an existing paper.

6.3 Existing question correction workflow

1. A verified contributor opens an existing question.
2. They select Request Correction.
3. They explain the problem and may attach a proposed replacement file.
4. The system creates a correction request linked to the existing question.
5. The original question remains published and unchanged.
6. An authorized admin reviews the request.
7. The admin may approve, reject, or request further information.
8. If approved, the official record is updated or a new file version is created.
9. The system records the reviewer, decision, timestamps, and previous version.
10. The requester can view the final status.

6.4 Admin editing workflow

An authorized admin can edit an official question directly from the dashboard.

Before saving, the backend must confirm the admin's role and department scope.

The system should preserve sufficient history to recover previous versions and determine who made each change.

6.5 Recommended status values

Use separate statuses for the official question and the proposed change.

Official question status

- published
- archived

Upload submission status

- pending_review
- published
- rejected

- needs_changes

Correction request status

- pending_review
- approved
- rejected
- needs_changes
- cancelled

A question's publication status must not be changed to pending_review merely because someone requested a correction. The original published version remains visible.

---

7. Authentication and EduMail Verification

7.1 Authentication

Public browsing does not require an account.

Users must authenticate before accessing contributor features. Authentication may use a secure email-based account system.

The chosen implementation should support:

- Registration.
- Login.
- Email verification.
- Logout.
- Password reset, if password-based authentication is used.
- Session expiration.
- Account suspension.
- Secure account recovery.

7.2 EduMail verification

The application must confirm both:

1. The email address belongs to an eligible NSTU email domain or identity system.

2. The user controls the mailbox, demonstrated through a verification link, one-time code, or trusted institutional authentication.

A matching email suffix alone is insufficient.

Before production, the project owner must confirm the official NSTU EduMail domain(s) and any institutional authentication requirements.

Do not hardcode an unverified domain assumption into production.

7.3 Verification lifecycle

The database should store:

- Email address.
- Verification timestamp.
- Verification method or provider.
- Account status.
- Relevant authentication-provider identifier.

The backend must reject contribution requests if the account is unverified, suspended, or otherwise ineligible.

---

8. Admin Roles and Department ID Logic

8.1 Two administrative roles

Super Admin

Global administrative scope across all departments.

Department Admin

Restricted to the department explicitly assigned to the administrator.

A Department Admin must not be able to change their own department assignment or elevate their own privileges through ordinary APIs.

8.2 Example student ID

Consider this example:

ASH2325006M

According to the project's illustrative ID format:

Segment	Example interpretation
ASH	Hall identifier
23	University session identifier
25	Department ID
006	Student's unique number
M	Gender identifier

Here, 25 represents the Institute of Information Technology (IIT), assuming this mapping is verified by the university or project administrator.

The project uses the department ID to determine department-specific administrative scope.

8.3 Do not trust a submitted ID by itself

The backend must not grant permissions merely because someone types an ID such as ASH2325006M.

Instead:

1. Authenticate the account.
2. Retrieve the verified user record.
3. Retrieve the administrator role and assigned department from trusted database records.
4. Confirm that the requested action is allowed.
5. Confirm that the target question belongs to the permitted department.
6. Execute the action only after all checks pass.

The department ID extracted from a student ID can be useful during onboarding or administrator assignment, but it is not sufficient proof of identity or administrative authority.

8.4 Recommended permission matrix

Action	Public	Verified contributor	Department Admin	Super Admin
Browse published papers	Yes	Yes	Yes	Yes

Download published papers	Yes	Yes	Yes	Yes
Submit a new paper	No	Yes	Yes	Yes
Request a correction	No	Yes	Yes	Yes
Edit official papers directly	No	No	Assigned department	All departments
Review correction requests	No	No	Assigned department	All departments
Manage departments globally	No	No	No	Yes
Manage admin assignments	No	No	No	Yes
View global audit logs	No	No	No	Yes

All privileged operations require server-side enforcement.

---

9. UI/UX and Glassmorphism Design System

9.1 Design objective

Hello NSTUian should look modern, academic, lightweight, and visually consistent.

Glassmorphism should be used to create frosted-glass cards, subtle transparency, blurred backgrounds, and restrained highlights.

The design must prioritize readability and performance over decorative effects.

9.2 Mobile-first requirements

Mobile phone users are the primary audience.

The interface must:

- Work on narrow mobile screens without horizontal overflow.
- Use responsive layouts for department and course cards.
- Provide comfortable touch targets.
- Keep search easy to access.

- Use readable font sizes.
- Avoid excessive blur and transparency.
- Provide clear loading, empty, success, and error states.
- Support keyboard navigation and accessible form labels.
- Provide fallbacks for browsers that do not support backdrop-filter.

9.3 Main public pages

Home

- Project logo/name.
- Search box.
- A–Z department cards.
- Quick navigation to recently added or popular papers, if implemented.
- Contribution entry point.
- Footer with project information.

Department page

- Department name and ID.
- Search within the department.
- University session selector.
- Available academic programs or course structures.

Session page

- University admission session.
- Semester selection.

Semester page

- Course cards sorted by course code or title.
- Course search.

Course page

- Course title and code.

- CT question sections.
- Semester/final question section.
- Available examination years/sessions.

Question page

- Question metadata.
- File preview.
- Download.
- Request Correction.

Authentication pages

- Registration.
- Login.
- Email verification.
- Password recovery, where applicable.

9.4 Admin dashboard pages

- Overview.
- Question management.
- Upload review queue for flagged submissions.
- Correction requests.
- Departments.
- Sessions and semesters.
- Courses.
- Users.
- Admin assignments.
- Audit logs.
- System settings.

The dashboard should clearly distinguish between a new submission requiring no manual review, a possible duplicate awaiting review, and a correction request requiring approval.

---

10. Recommended Technical Architecture

10.1 Proposed technology stack

The following is the recommended baseline for the initial implementation.

Layer	Technology	Purpose
Frontend	React + Vite	Public website and admin interface
Styling	CSS or Tailwind CSS	Responsive glassmorphism design
Backend	Node.js + Express.js	Business logic and APIs
Database	PostgreSQL	Academic data, accounts, permissions, and history
File storage	Private object storage, such as Supabase Storage	Original question files and proposed replacements
Authentication	Secure server-managed sessions or a suitable authentication provider	Identity and access
Email	Configured SMTP or transactional email provider	Verification and notifications
Deployment	Frontend hosting, backend hosting, managed PostgreSQL, and object storage	Production operation

This stack is a proposal, not a claim that any infrastructure has already been configured.

10.2 System components

Frontend

Responsible for:

- Rendering pages.

- Navigation.
- Search and filtering.
- Forms.
- Question previews.
- Downloads.
- Authentication interfaces.
- Admin dashboard interfaces.

The frontend must not contain secrets or be trusted to enforce permissions.

Backend

Responsible for:

- Authentication checks.
- EduMail verification checks.
- Role and department authorization.
- Academic data validation.
- Upload validation.
- Duplicate detection.
- Automatic publication.
- Correction approval workflows.
- Audit logging.
- Secure file access.

Database

Responsible for:

- Storing structured academic information.
- Connecting departments, courses, sessions, and papers.
- Storing user and permission records.
- Tracking upload and correction requests.

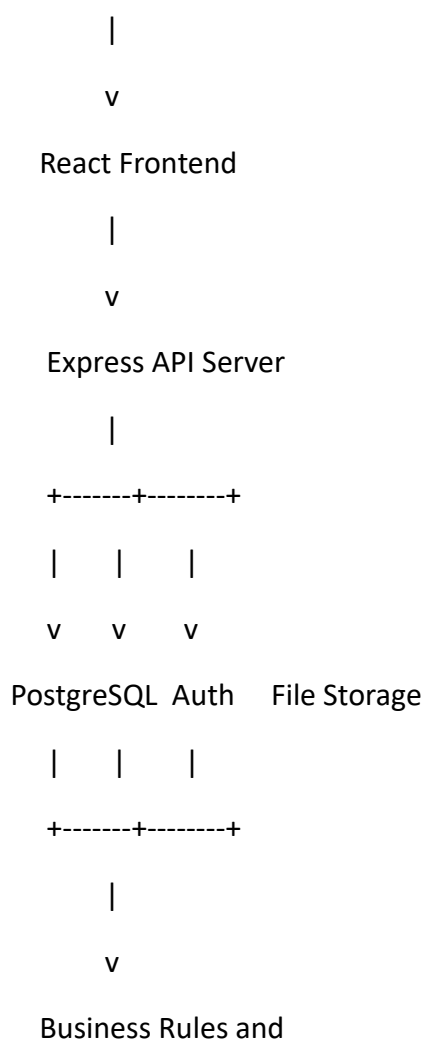
- Maintaining publication statuses and version history.

File storage

Responsible for:

- Storing original files.
- Keeping proposed replacements separate from published files.
- Restricting access to pending files.
- Providing controlled delivery of published files.
- Supporting file retention and deletion policies.

10.3 High-level architecture



Authorization Checks

Administrative Dashboard

|

v

Express API Server

|

v

Role + Department Permission Checks

|

v

Approved Administrative Actions

Every protected action passes through backend validation and authorization before the database or official file record is changed.

---

11. Database Design

Use PostgreSQL with primary keys, foreign keys, appropriate indexes, timestamps, and database constraints.

The schema below is a logical design. Exact SQL migrations should be created during implementation.

11.1 departments

Purpose: Store NSTU departments.

Fields:

- id: Primary key.
- department_code: Unique internal department ID.
- name: Official department name.
- abbreviation: Optional abbreviation.

- description: Optional description.
- is_active: Whether the department is active.
- created_at
- updated_at

The department code must be unique and must not be confused with the internal database primary key.

11.2 university_sessions

Purpose: Store admission batches.

Fields:

- id
- session_label: For example, 2022-2023.
- start_year
- end_year
- is_active
- created_at

Use consistent session formatting throughout the application.

11.3 academic_programs

Purpose: Represent the academic structure where a department has multiple programs or curricula.

Fields:

- id
- department_id
- program_name
- program_code
- is_active

If the initial implementation confirms that each department needs only one program structure, the schema can be simplified. The design should not silently assume that every academic program has identical course requirements.

11.4 semesters

Purpose: Store semester definitions.

Fields:

- id
- program_id
- semester_number: 1–8.
- semester_name
- created_at

Constraint: semester_number must be between 1 and 8.

If semester definitions are shared across admission sessions, keep them as reusable records. Session-specific curricula should be represented separately when needed.

11.5 courses

Fields:

- id
- program_id
- semester_id
- course_code
- course_title
- credit, optional
- description, optional
- is_active
- created_at
- updated_at

Use suitable uniqueness constraints to avoid accidental duplicate course records within the same academic structure.

11.6 users

Fields:

- id
- email
- email_verified_at, nullable
- display_name
- account_status
- auth_provider
- auth_provider_user_id, when applicable
- created_at
- updated_at

Store password hashes only if password authentication is implemented directly. Prefer a maintained authentication library or trusted provider instead of inventing authentication mechanisms.

11.7 admin_permissions

Fields:

- id
- user_id
- role: super_admin or department_admin.
- department_id, nullable for Super Admin.
- assigned_by
- created_at
- updated_at

Rules:

- Super Admin has global scope.

- Department Admin must have a valid assigned department.
- Ordinary contributors have no admin permissions.
- Assignment changes must be auditable.
- Role assignments must not be self-service.

A user may have multiple department assignments only if the project owner explicitly enables that policy. The default specification assigns a Department Admin to one department.

11.8 exam_sessions

Purpose: Distinguish examination periods from university admission batches.

Fields:

- id
- label: For example, 2025.
- academic_year, optional.
- exam_date, optional.
- created_at

Use this table for examination metadata, not as a replacement for university admission sessions.

11.9 exam_types

Fields:

- id
- code: For example, CT or SEMESTER_FINAL.
- name
- requires_ct_number
- is_active

This allows the project to add examination categories without adding a new database column for every type.

11.10 question_papers

Purpose: Store the official question-paper records.

Fields:

- id
- department_id
- course_id
- university_session_id, nullable where appropriate.
- exam_session_id
- exam_type_id
- ct_number, nullable.
- set_name, nullable.
- title, nullable.
- description, nullable.
- publication_status: published or archived.
- current_version_id, nullable until a version is created.
- created_by
- created_at
- updated_at

Validation rules:

- The selected course must belong to the appropriate department/program.
- CT number is required for CT papers.
- CT number must be valid for the supported CT configuration.
- Set names must be validated.
- Examination metadata must be valid.
- Public APIs return only published papers.
- Archiving a paper removes it from ordinary public search without erasing its audit history.

11.11 question_paper_versions

Purpose: Preserve original files and official revisions.

Fields:

- id
- question_paper_id
- version_number
- storage_key
- original_filename
- mime_type
- file_size
- sha256_hash
- created_by
- created_at
- change_summary, nullable

Each official replacement creates a new version instead of silently destroying the previous file.

The question-paper record identifies the current version.

11.12 upload_requests

Purpose: Track first-time user submissions and possible duplicates.

Fields:

- id
- submitted_by
- department_id
- course_id
- university_session_id, nullable.
- exam_session_id
- exam_type_id
- ct_number, nullable.
- set_name, nullable.

- temporary_storage_key
- original_filename
- mime_type
- file_size
- sha256_hash
- status
- published_question_paper_id, nullable.
- reviewed_by, nullable.
- reviewed_at, nullable.
- rejection_reason, nullable.
- created_at

The backend may publish a valid new submission automatically and record the resulting official paper. A flagged duplicate should remain unpublished until resolved.

11.13 correction_requests

Purpose: Store proposed changes to existing papers.

Fields:

- id
- question_paper_id
- requested_by
- description
- proposed_storage_key, nullable.
- status
- reviewed_by, nullable.
- reviewed_at, nullable.
- review_notes, nullable.
- resulting_version_id, nullable.

- created_at
- updated_at

The proposed file must remain separate from the current official file until approval.

11.14 audit_logs

Fields:

- id
- actor_user_id
- action
- entity_type
- entity_id
- department_id, nullable.
- details, structured JSON where appropriate.
- created_at

Audit logs should record administrative actions, approval decisions, permission changes, publication events, and file replacements.

Avoid storing passwords, authentication tokens, or other secrets in logs.

11.15 Database relationship summary

Department

|

+-- Academic Programs

|

+-- Semesters (1 to 8)

|

+-- Courses

|

+-- Question Papers

|

+-- Paper Versions

|

+-- Correction Requests

University Sessions

|

+-- Academic context for courses and papers

Exam Sessions

|

+-- Question Papers

Users

|

+-- Upload Requests

+-- Correction Requests

+-- Admin Permissions

+-- Audit Logs

The actual foreign keys and relationships must be defined in the database migrations. This diagram is conceptual, not executable SQL.

---

12. API Design

All APIs should use consistent response structures, validation, error handling, and pagination where needed.

Use a versioned base path such as /api/v1.

12.1 Public APIs

Method	Endpoint	Purpose
GET	/departments	List active departments in A–Z order
GET	/departments/:id	Get department details
GET	/departments/:id/sessions	Get available university sessions
GET	/departments/:id/semesters	Get the supported academic semesters
GET	/departments/:id/courses	List courses
GET	/courses/:id	Get course details
GET	/courses/:id/questions	List published question papers
GET	/questions/:id	Get a published question's metadata
GET	/questions/:id/download	Deliver a published question file
GET	/search	Search public departments, courses, and papers

All public question endpoints must filter out unpublished and archived records as appropriate.

12.2 Authentication APIs

Exact endpoints depend on the selected authentication provider.

Possible application routes include:

- POST /auth/register
- POST /auth/login
- POST /auth/logout
- POST /auth/verify-email
- POST /auth/resend-verification
- GET /auth/me

If a provider manages authentication and email verification, use its supported SDK/API and map verified identities into the application's user model.

Never trust a frontend-only emailVerified flag.

12.3 Contributor APIs

Method	Endpoint	Purpose
POST	/uploads	Submit a new question paper
GET	/uploads/my	List the user's submissions
GET	/correction-requests/my	List the user's correction requests
POST	/questions/:id/correction-requests	Propose a correction
GET	/correction-requests/:id	View an authorized request

The backend decides whether a submission is a new paper, a possible duplicate, or an invalid attempt to modify existing content.

Do not expose an API that lets contributors directly update official question-paper records.

12.4 Admin APIs

Possible endpoints include:

- GET /admin/dashboard
- GET /admin/uploads
- GET /admin/correction-requests
- POST /admin/uploads/:id/approve
- POST /admin/uploads/:id/reject
- POST /admin/correction-requests/:id/approve
- POST /admin/correction-requests/:id/reject
- PATCH /admin/questions/:id
- POST /admin/questions/:id/versions
- POST /admin/questions/:id/archive
- POST /admin/departments
- PATCH /admin/departments/:id
- POST /admin/courses
- PATCH /admin/courses/:id

- GET /admin/audit-logs
- GET /admin/admins
- POST /admin/admins

Every admin route must validate the user's role and the requested resource's department.

Endpoint names are proposed conventions. The implementation should document its final routes and schemas.

12.5 Standard error responses

Use clear, predictable responses, for example:

- 400 Bad Request: Invalid input.
- 401 Unauthorized: Authentication is required.
- 403 Forbidden: The user lacks the necessary permission.
- 404 Not Found: The requested resource does not exist or is not accessible.
- 409 Conflict: Duplicate or conflicting operation.
- 413 Payload Too Large: File exceeds the configured limit.
- 415 Unsupported Media Type: Unsupported or invalid file type.
- 422 Unprocessable Entity: Valid request format but invalid business data.
- 429 Too Many Requests: Rate limit exceeded.
- 500 Internal Server Error: Unexpected server failure.

Error messages must not reveal private records or internal security details.

---

13. Security Requirements

13.1 Authorization

Authorization must be enforced on the server for every protected operation.

A hidden button, disabled form, or frontend route guard is not a security control by itself.

For a Department Admin's edit request, the backend must:

1. Identify the authenticated user.

2. Retrieve the user's actual admin role and assigned department.
3. Retrieve the target question.
4. Compare the question's department with the authorized department.
5. Validate the requested modification.
6. Save the change only if all checks succeed.
7. Record the operation in the audit log.

13.2 File upload security

The application must:

- Allow only configured file types.
- Inspect actual file content rather than relying only on extensions.
- Apply file-size and upload-rate limits.
- Generate unpredictable storage keys.
- Prevent path traversal and filename-based attacks.
- Keep pending files private.
- Never execute uploaded files.
- Avoid serving untrusted HTML or JavaScript as active application content.
- Use safe download headers and content types.
- Protect file-processing tools against malicious documents.
- Clean up abandoned temporary files according to a documented retention policy.

13.3 Authentication security

Use HTTPS in production.

For cookie-based authentication, configure secure cookies and appropriate CSRF protection. For token-based authentication, use a well-reviewed authentication design and avoid storing long-lived credentials in insecure browser storage.

Apply rate limits to login, verification, upload, and correction endpoints.

13.4 Database security

- Use parameterized queries or a safe ORM.

- Do not concatenate untrusted input into SQL.
- Use least-privilege database credentials.
- Validate foreign-key relationships.
- Protect production secrets using environment variables or a secret-management service.
- Keep database backups and test recovery procedures.
- Restrict direct production database access.

13.5 Auditability

Record who performed each important administrative action, what was changed, when it happened, and the result.

A Super Admin should be able to investigate unauthorized attempts and restore previous official file versions when appropriate.

---

14. Search and Performance

Search should be available from the landing page and from relevant department/course pages.

Initial search features should include:

- Department-name search.
- Department-code search.
- Course-code search.
- Course-title search.
- Filtering by university session.
- Filtering by semester.
- Filtering by examination type.
- Filtering by examination year/session.

Search results must show only published papers to public users.

Use database indexes for common lookup fields and foreign keys. Add full-text search if required after evaluating the initial dataset and performance.

Use pagination for large result sets.

Do not load every department's entire question collection into the browser at once.

For mobile devices, optimize image previews, use appropriate loading behavior, and avoid downloading original high-resolution files until requested.

---

15. Accessibility and Compatibility

The website should:

- Support current major desktop and mobile browsers.
- Work on small Android and iOS screens.
- Provide keyboard-accessible navigation.
- Use semantic HTML.
- Label all input fields.
- Show validation errors clearly.
- Maintain sufficient color contrast.
- Avoid communicating status through color alone.
- Provide a usable fallback when glass effects are unsupported.
- Handle unavailable previews gracefully.

Public pages should not require JavaScript-heavy admin modules to load before users can access basic browsing features.

---

16. Testing Requirements

Testing is mandatory before production release.

16.1 Functional testing

Verify that:

- Departments are displayed in A–Z order.
- Search returns relevant results.
- Academic navigation works.
- Each program supports the required eight semesters.

- Courses display the correct papers.
- CT categories and sets work correctly.
- Semester examination questions are organized correctly.
- Public visitors can view and download published papers.
- Unpublished papers do not appear in public search.

16.2 Upload workflow testing

Test:

- Verified user uploads a valid new paper: automatically published.
- Unverified user attempts to upload: rejected.
- Suspended user attempts to upload: rejected.
- Valid but potentially duplicate paper: flagged for review.
- Invalid or malicious file: rejected or quarantined.
- Supported document without preview support: safe download fallback.
- Uploaded file exceeding the size limit: rejected.
- User uploads a paper for another department: allowed if otherwise valid.

16.3 Correction workflow testing

Test:

- Verified user submits a correction request.
- Existing paper remains unchanged while the request is pending.
- Unauthorized user cannot edit the official record.
- Authorized Department Admin can approve changes in their own department.
- Department Admin cannot approve or edit another department's paper.
- Super Admin can manage all departments.
- Rejected changes do not alter the official paper.
- Approved replacements create an auditable new version.

16.4 Security testing

Test for:

- Unauthorized API access.
- Department ID manipulation.
- Privilege escalation.
- Cross-site scripting.
- SQL injection.
- Cross-site request forgery where relevant.
- Malicious file uploads.
- Unsafe file delivery.
- Brute-force attempts.
- Rate-limit bypass.
- Accidental publication of pending content.

16.5 Responsive testing

Test on:

- Small mobile phones.
- Larger mobile phones.
- Tablets.
- Desktop screens.

Verify there is no unwanted horizontal scrolling, clipped text, inaccessible navigation, or unusable upload forms.

---

17. Deployment and Operations

17.1 Environments

Maintain separate environments for:

- Local development.
- Testing/staging.

- Production.

Do not use production credentials in the source code or commit secret files to version control.

17.2 Deployment requirements

The production deployment must include:

- Frontend hosting.
- Backend hosting.
- Managed PostgreSQL or an appropriately secured database server.
- Object storage for files.
- HTTPS.
- Authentication and email-verification configuration.
- Environment-specific settings.
- Database migrations.
- Logging and error monitoring.
- Backup and recovery procedures.

17.3 Suggested environment variables

Names depend on the chosen providers, but may include:

NODE_ENV=

PORT=

DATABASE_URL=

APP_BASE_URL=

SESSION_SECRET=

AUTH_PROVIDER_URL=

AUTH_PROVIDER_PUBLIC_KEY=

AUTH_PROVIDER_SERVER_KEY=

EMAIL_PROVIDER_API_KEY=

OBJECT_STORAGE_ENDPOINT=

OBJECT_STORAGE_BUCKET=

OBJECT_STORAGE_ACCESS_KEY=

OBJECT_STORAGE_SECRET_KEY=

MAX_UPLOAD_SIZE_MB=

Use only the variables required by the final architecture. Never expose server-side secrets through frontend build-time variables or public client bundles.

17.4 Operational safeguards

- Apply database migrations in a controlled deployment process.
- Back up the database and important files.
- Test restoration periodically.
- Monitor server errors and failed uploads.
- Monitor storage capacity.
- Keep dependencies updated.
- Document procedures for account recovery and admin reassignment.

---

18. Suggested Project Folder Structure

The following is a proposed structure for a React + Express + PostgreSQL implementation.

hello-nstuian/

|

+-- client/

| +-- public/

| +-- src/

| +-- assets/

| +-- components/

| +-- layouts/

| +-- pages/

```
|       |   +-- Home/
|       |   +-- Departments/
|       |   +-- Sessions/
|       |   +-- Semesters/
|       |   +-- Courses/
|       |   +-- Questions/
|       |   +-- Auth/
|       |   +-- Admin/
|       +-- features/
|       |   +-- search/
|       |   +-- uploads/
|       |   +-- corrections/
|       |   +-- departments/
|       +-- services/
|       +-- hooks/
|       +-- styles/
|       +-- utils/
|       +-- App.jsx
|       +-- main.jsx
|
+-- server/
|   +-- src/
|       +-- config/
|       +-- middleware/
|       +-- modules/
|       |   +-- auth/
```

```
|       |   +-- users/
|       |   +-- departments/
|       |   +-- sessions/
|       |   +-- semesters/
|       |   +-- courses/
|       |   +-- questions/
|       |   +-- uploads/
|       |   +-- corrections/
|       |   +-- admins/
|       |   +-- audit/
|       +-- routes/
|       +-- services/
|       +-- validators/
|       +-- utils/
|       +-- app.js
|       +-- server.js
|   +-- tests/
|
+-- database/
|   +-- migrations/
|   +-- seeds/
|   +-- schema/
|
+-- docs/
|   +-- MASTER_PROJECT_DOCUMENTATION.md
|   +-- API_SPECIFICATION.md
```

| +-- DATABASE_DESIGN.md

| +-- TEST_PLAN.md

| +-- DEPLOYMENT_GUIDE.md

|

+-- .gitignore

+-- README.md

This is a starting point, not a requirement to create every directory immediately. The developer should keep modules organized around responsibilities and avoid unnecessary complexity.

---

19. Development Roadmap

Phase 1: Requirements and foundation

Tasks:

- Confirm the official department list and IDs.
- Confirm university session and course structures.
- Finalize the design system.
- Create the repository and project structure.
- Configure development environments.

Deliverable: A runnable project skeleton and agreed data model.

Phase 2: Public frontend

Tasks:

- Build the landing page.
- Create A–Z department cards.
- Implement search.
- Build session, semester, and course pages.
- Create question detail and download interfaces.
- Implement responsive glassmorphism styling.

Deliverable: A responsive public interface connected to test data.

Phase 3: Database and backend

Tasks:

- Create database migrations.
- Implement academic CRUD operations.
- Implement public APIs.
- Implement search and filtering.
- Add validation and error handling.

Deliverable: Functional academic browsing backed by PostgreSQL.

Phase 4: Authentication and authorization

Tasks:

- Implement authentication.
- Integrate EduMail verification.
- Implement contributor eligibility checks.
- Implement Super Admin and Department Admin roles.
- Add server-side permission checks.

Deliverable: Secure access control.

Phase 5: File upload and publication

Tasks:

- Integrate object storage.
- Implement upload validation.
- Implement duplicate detection.
- Implement automatic publication of valid new papers.
- Implement safe previews and downloads.

Deliverable: Working question contribution system.

Phase 6: Correction and admin workflows

Tasks:

- Build the admin dashboard.
- Implement correction requests.
- Implement review and approval.
- Implement version history.
- Implement audit logs.
- Implement department-specific administration.

Deliverable: Complete management and moderation workflows.

Phase 7: Testing and deployment

Tasks:

- Run functional and security tests.
- Verify responsive behavior.
- Test permissions with multiple departments.
- Configure production hosting.
- Configure backups and monitoring.
- Prepare user and admin documentation.

Deliverable: A tested, deployable initial release.

---

20. Acceptance Criteria

The project is ready for initial release only when the following requirements are satisfied.

Public experience

- The home page works on mobile and desktop.
- Departments are sorted alphabetically.
- Search and academic navigation work.
- All eight semesters can be represented for each supported program.
- CT questions, CT sets, and semester questions are supported.

- Public visitors can view and download published questions without login.

Contribution system

- Uploading requires authentication and verified NSTU EduMail.
- Verified contributors can submit papers for any department.
- Valid new and non-duplicate papers are automatically published.
- Potential duplicates are not automatically published.
- Supported file types are validated.
- Unsafe files are rejected or quarantined.

Correction system

- Verified users can request corrections.
- Regular users cannot directly edit official papers.
- The original paper remains available while a correction is pending.
- Authorized admins can approve or reject proposed changes.
- Approved replacements create an auditable new version.

Administration

- Super Admin can manage all departments.
- Department Admin can manage only their assigned department.
- Department access is verified by the backend.
- Admin assignment changes are auditable.
- Important actions are recorded in audit logs.

Production readiness

- Database constraints and migrations are in place.
- Authentication and file-delivery security have been tested.
- Error handling and logging work.
- Backup and recovery procedures are documented.
- The application can be deployed using documented instructions.

---

21. Decisions Requiring Confirmation Before Production

The core requirements are defined, but a few operational details must be confirmed rather than invented.

1. **Official department list:** Confirm every department's official name, abbreviation, and department ID.
2. **EduMail domain:** Confirm the official eligible NSTU email domain(s) or identity provider.
3. **File-size limit:** Choose a suitable maximum upload size and storage quota.
4. **Academic curricula:** Confirm whether every department has exactly one eight-semester structure or whether programs and curricula differ.
5. **Duplicate handling:** Confirm who reviews flagged possible duplicates and whether a contributor can cancel a pending submission.
6. **Admin provisioning:** Decide how the first Super Admin is securely created.
7. **Retention policy:** Define how archived questions, rejected submissions, and temporary files are retained or deleted.
8. **Hosting and budget:** Choose production hosting and storage providers based on cost, reliability, and expected usage.

These details should be configurable wherever practical.

Do not fabricate official NSTU data, real department IDs, verified email domains, or administrator accounts.

---

22. Master Instructions for an AI Developer

The following section is intended to be provided to an AI coding agent along with this documentation.

Role

Act as a senior full-stack software engineer, database designer, security engineer, UI/UX developer, and test engineer.

Build the Hello NSTUian application according to this document.

Required behavior

1. Read the complete documentation before implementing the application.
2. Treat the business rules and acceptance criteria as authoritative.
3. Use the selected technology stack consistently unless a justified change is approved.
4. Inspect the existing repository before creating or replacing files.
5. Never assume that an existing project is empty.
6. Preserve useful existing code and explain any required migration.
7. Implement the frontend, backend, database, storage integration, authentication, authorization, and administrative workflows as a connected system.
8. Use real API calls and persistent database records; do not substitute mock data for completed production functionality.
9. Keep secrets out of source code and frontend bundles.
10. Validate all user input on the backend.
11. Enforce all admin permissions on the server.
12. Keep pending files private and publish only eligible submissions.
13. Automatically publish a valid, genuinely new question after the required checks.
14. Require admin approval before applying any user-requested modification to an existing question.
15. Maintain audit history and official file versions.
16. Implement responsive, accessible glassmorphism UI.
17. Add automated tests for business rules, API behavior, and authorization boundaries.
18. Document setup, migrations, environment variables, testing, and deployment.
19. Never claim that a feature works unless it has been implemented and tested.
20. If a required external credential or institutional detail is unavailable, implement the appropriate integration boundary, document the missing configuration, and continue with the remaining work.

Implementation workflow

Work in small, verifiable stages:

Stage A: Inspect repository, identify existing files, and prepare an implementation plan.

Stage B: Establish the database schema and migrations.

Stage C: Implement the backend and authorization middleware.

Stage D: Implement public browsing and search.

Stage E: Implement authentication and EduMail verification.

Stage F: Implement secure uploads, duplicate detection, and automatic publication.

Stage G: Implement correction requests, admin approval, versioning, and audit logs.

Stage H: Implement and connect the complete admin dashboard.

Stage I: Run tests, fix errors, verify mobile responsiveness, and document deployment.

After each stage, report:

- What was implemented.
- Which files were created or changed.
- Which tests were run.
- Any remaining issues.
- The next implementation stage.

Do not stop after generating a plan or static UI. Continue toward a complete working application within the available environment and permissions.

Final delivery requirements

Deliver:

- Complete source code.
- Database migrations and seed scripts.
- .env.example with placeholder values only.
- README with setup instructions.
- API documentation.
- Database documentation.
- Automated tests.

- Admin usage guide.
- Deployment guide.
- A list of incomplete features and required external configuration.

The final application must follow the acceptance criteria in Section 20.

---

23. Final Product Definition

Hello NSTUian is a public academic question bank for NSTU, designed primarily for mobile users.

Its central workflow is:

Find questions easily → contribute new questions conveniently → protect existing papers through controlled modifications.

Public access remains open. Verified NSTU EduMail users can contribute to any department. Valid new papers can be published automatically after backend validation, while changes to existing official papers require authorized admin approval.

The Super Admin manages the entire platform, and Department Admins are restricted to their assigned departments. The database, file-storage layer, APIs, and UI must work together to enforce these rules.

This document is the project's baseline specification. Implementation details may evolve, but changes must preserve the agreed business rules, security boundaries, and acceptance criteria unless the project owner explicitly approves a change.

End of Master Project Documentation — Hello NSTUian, Version 1.0